```
d3.select("#controls").append("button").on("click", function (){
                      zoomButton("out")}).html("Zoom Out");
```

With this kind of styling and interactivity in place, you can make a map for most any application. Zooming and panning is important for maps because users expect to be able to zoom in and out, and they also expect the details of the map to change when they do so. In that way, geospatial is one of the most powerful forms of information visualization because users have a high level of literacy when it comes to reading and interacting with maps. But users also expect a map to have certain features and functionality, and when those are missing they think it's broken. Make sure that when you create your map, it either includes this functionality or you have a good reason to leave it out.

## 7.3 Advanced mapping

We've covered the aspects of creating maps that you'll likely end up using with all your maps. You could explore many variations. You may want to scale your `<circle>` elements based on population, or use `<g>` elements so that you can also provide labels like we did earlier. But if you're making a map, it will probably have polygons and points and take advantage of bounding boxes or centroids, and will likely be tied to a zoom behavior. The exciting thing about D3 is that it lets you explore more complex ways of representing geography, with a little more effort.

### 7.3.1 Creating and rotating globes

We'll do only one thing in 3D in this entire book, and that's create a globe. We don't need to load three.js or learn WebGL. Instead, we'll take advantage of a trick of one of the geographic projections available in D3: the orthographic projection, which renders geographic data as it would appear from a distant point viewing the entire globe. We need to update our projection to refer to the orthographic projection and have a slightly different scale.

**Listing 7.12 Creating a simple globe**

```
projection = d3.geo.orthographic()
    .scale(200)
    .translate([width / 2, height / 2])
    .center([0,0]);
```

With this new projection, you can see what looks like a globe in figure 7.11.

To make it rotate, we need to use `d3.mouse`, which returns the current position of the mouse on the SVG canvas. Pair this with event listeners to turn on and off a mousemove listener on the canvas. This simulates dragging the globe, which we'll use only to rotate it along the x-axis. Because we're introducing new behavior and it's been a while since we looked at the full code, the following listing has the entire code for creating the globe.